

Лабораторная работа №5

Основы информационной безопасности

Верниковская Е. А., НПИбд-01-23

17 апреля 2025

Российский университет дружбы народов, Москва, Россия

Вводная часть

Целью данной работы является изучение механизмов изменения идентификаторов, применения SetUID- и Sticky-битов. Получение практических навыков работы в консоли с дополнительными атрибутами. Рассмотрение работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

Выполнение лабораторной работы

Подготовка лабораторного стенда

Для лабораторной работы необходимо проверить, установлен ли компилятор gcc. Проверим это с помощью команды `gcc -v`. Также осуществим отключение системы запретов с помощью `sudo setenforce 0` и проверим командой `getenforce`, что выводится Permissive (рис. 1)

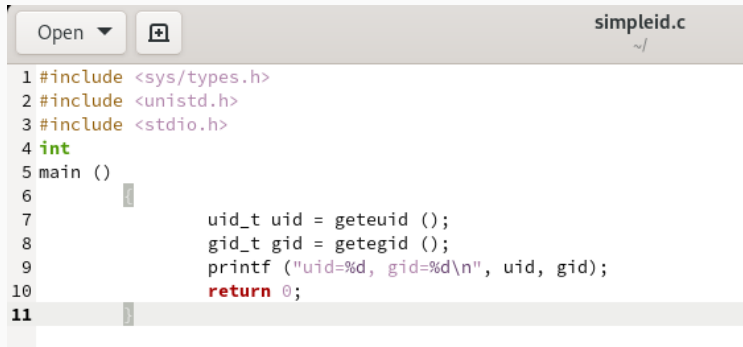
```
[eavernikovskaya@eavernikovskaya ~]$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/libexec/gcc/x86_64-redhat-linux/11/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-redhat-linux
Configured with: ../configure --enable-bootstrap --enable-host-pie --enable-host-bind-now --enable-languages=c,c++,for
tran,lto --prefix=/usr --mandir=/usr/share/man --infodir=/usr/share/info --with-bugurl=https://bugs.rockylinux.org/ --
enable-shared --enable-threads=posix --enable-checking=release --with-system-zlib --enable-_cxa_atexit --disable-libu
nwind-exceptions --enable-gnu-unique-object --enable-linker-build-id --with-gcc-major-version-only --enable-plugin --e
nable-initfini-array --without-isl --enable-multilib --with-linker-hash-style=gnu --enable-offload-targets=nvptx-none
--without-cuda-driver --enable-gnu-indirect-function --enable-cet --with-tune=generic --with-arch_64=x86-64-v2 --with-
arch_32=x86-64 --build=x86_64-redhat-linux --with-build-config=bootstrap-lto --enable-link-serialization=1
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 11.5.0 20240719 (Red Hat 11.5.0-2) (GCC)
[eavernikovskaya@eavernikovskaya ~]$ sudo setenforce 0
[sudo] password for eavernikovskaya:
[eavernikovskaya@eavernikovskaya ~]$ getenforce
Permissive
[eavernikovskaya@eavernikovskaya ~]$
```

Рис. 1: Подготовка к лабораторной работе

Войдём в систему от имени пользователя guest. И создадим программу simpleid.c (рис. 2), (рис. 3)

```
[eavernikovskaya@eavernikovskaya ~]$ su guest
Password:
[guest@eavernikovskaya eavernikovskaya]$ cd ~
[guest@eavernikovskaya ~]$ touch simpleid.c
[guest@eavernikovskaya ~]$ gedit simpleid.c
```

Рис. 2: Создание и открытие файла simpleid.c



```
1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdio.h>
4 int
5 main ()
6 {
7     uid_t uid = geteuid ();
8     gid_t gid = getegid ();
9     printf ("uid=%d, gid=%d\n", uid, gid);
10    return 0;
11 }
```

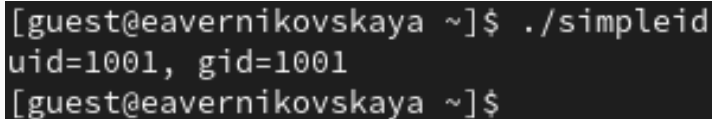
Рис. 3: Редактирование файла simpleid.c

Скомпилируем программу и убедимся, что файл программы создан: `gcc simpleid.c -o simpleid` (рис. 4)

```
[guest@eavernikovskaya ~]$ gcc simpleid.c -o simpleid
[guest@eavernikovskaya ~]$ ls
Desktop  dir2  Documents  file4  Pictures  simpleid  Templates
dir1     dir3  Downloads  Music  Public    simpleid.c  Videos
[guest@eavernikovskaya ~]$
```

Рис. 4: Компиляция программы `simpleid.c`

Выполним программу `simpleid`: `./simpleid`. В выводе файла выписаны номера пользователя и групп, от вывода при вводе `id`, они отличаются только тем, что информации меньше (рис. 5)

A terminal window with a dark background and light gray text. The prompt is [guest@eavernikovskaya ~]\$ and the command ./simpleid has been entered. The output is uid=1001, gid=1001. The prompt is now [guest@eavernikovskaya ~]\$.

```
[guest@eavernikovskaya ~]$ ./simpleid
uid=1001, gid=1001
[guest@eavernikovskaya ~]$
```

Рис. 5: Выполнение программы `simpleid`

Выполните системную программу `id`: *id* (рис. 6)

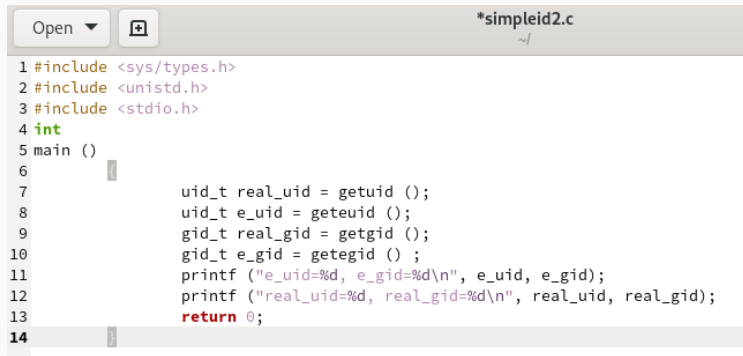
```
[guest@eavernikovskaya ~]$ id  
uid=1001(guest) gid=1001(guest) groups=1001(guest) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023  
[guest@eavernikovskaya ~]$
```

Рис. 6: Выполнение системной программы `id`

Далее усложним программу, добавив вывод действительных идентификаторов. Создадим программу под названием simpleid2.c (рис. 7), (рис. 8)

```
[guest@eavernikovskaya ~]$ cp simpleid.c simpleid2.c  
[guest@eavernikovskaya ~]$ gedit simpleid2.c
```

Рис. 7: Создание и открытие файла simpleid2.c



```
Open ▾  *simpleid2.c  
~/  
1 #include <sys/types.h>  
2 #include <unistd.h>  
3 #include <stdio.h>  
4 int  
5 main ()  
6  
7     uid_t real_uid = getuid ();  
8     uid_t e_uid = geteuid ();  
9     gid_t real_gid = getgid ();  
10    gid_t e_gid = getegid ();  
11    printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid);  
12    printf ("real_uid=%d, real_gid=%d\n", real_uid, real_gid);  
13    return 0;  
14
```

Рис. 8: Редактирование файла simpleid2.c

Скомпилируем и запустим `simpleid2.c`: `gcc simpleid2.c -o simpleid2` и `./simpleid2` (рис. 9), (рис. 10)

```
[guest@eavernikovskaya ~]$ gcc simpleid2.c -o simpleid2
[guest@eavernikovskaya ~]$ ls
Desktop  dir2  Documents  file4  Pictures  simpleid  simpleid2.c  Templates
dir1     dir3  Downloads  Music  Public   simpleid2  simpleid.c   Videos
[guest@eavernikovskaya ~]$
```

Рис. 9: Компиляция программы `simpleid2.c`

```
[guest@eavernikovskaya ~]$ ./simpleid2  
e_uid=1001, e_gid=1001  
real_uid=1001, real_gid=1001  
[guest@eavernikovskaya ~]$
```

Рис. 10: Выполнение программы simpleid2

От имени суперпользователя выполним команды: *chown root:guest /home/guest/simpleid2* и *chmod u+s /home/guest/simpleid2*. Также выполним проверку правильности установки новых атрибутов и смены владельца файла *simpleid2*: *ls -l simpleid2*. С помощью *chown* мы меняем владельца файла на суперпользователя, а с помощью *chmod* меняем права доступа (рис. 11)

```
[eavernikovskaya@eavernikovskaya ~]$ sudo chown root:guest /home/guest/simpleid2
[sudo] password for eavernikovskaya:
[eavernikovskaya@eavernikovskaya ~]$ sudo chmod u+s /home/guest/simpleid2
[eavernikovskaya@eavernikovskaya ~]$ ls -l /home/guest/simpleid2
ls: cannot access '/home/guest/simpleid2': Permission denied
[eavernikovskaya@eavernikovskaya ~]$ sudo ls -l /home/guest/simpleid2
-rwsr-xr-x. 1 root guest 17656 Apr 16 15:23 /home/guest/simpleid2
[eavernikovskaya@eavernikovskaya ~]$
```

Рис. 11: Смена владельца файла и прав доступа к файлу *simpleid2*

Запустим `simpleid2` и `id`: `./simpleid2` и `id`. Наша команда снова вывела только ограниченное количество информации (рис. 12)

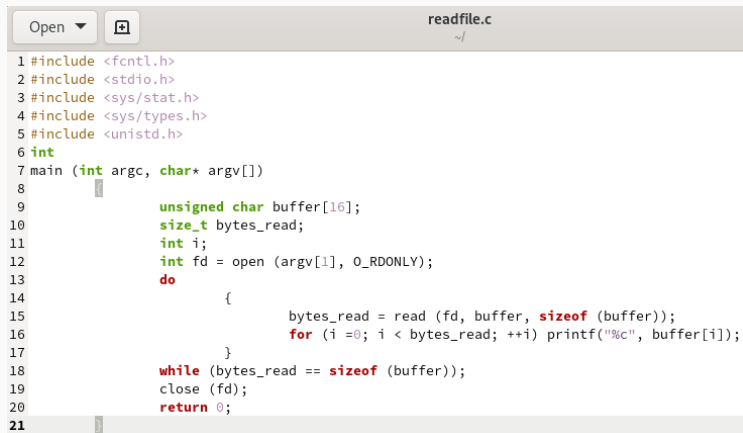
```
[eavernikovskaya@eavernikovskaya ~]$ sudo /home/guest/simpleid2
e_uid=0, e_gid=0
real_uid=0, real_gid=0
[eavernikovskaya@eavernikovskaya ~]$ id
uid=1000(eavernikovskaya) gid=1000(eavernikovskaya) groups=1000(eavernikovskaya),10(wheel) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
[eavernikovskaya@eavernikovskaya ~]$ sudo id
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
[eavernikovskaya@eavernikovskaya ~]$
```

Рис. 12: Запуск `simpleid2` и `id`

Создадим программу readfile.c (рис. 13), (рис. 14)

```
[guest@eavernikovskaya ~]$ touch readfile.c  
[guest@eavernikovskaya ~]$ gedit readfile.c
```

Рис. 13: Создание и открытие файла readfile.c



```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <sys/stat.h>
4 #include <sys/types.h>
5 #include <unistd.h>
6 int
7 main (int argc, char* argv[])
8 {
9     unsigned char buffer[16];
10    size_t bytes_read;
11    int i;
12    int fd = open (argv[1], O_RDONLY);
13    do
14    {
15        bytes_read = read (fd, buffer, sizeof (buffer));
16        for (i = 0; i < bytes_read; ++i) printf("%c", buffer[i]);
17    }
18    while (bytes_read == sizeof (buffer));
19    close (fd);
20    return 0;
21 }
```

Рис. 14: Редактирование файла readfile.c

Откомпилируем её: `gcc readfile.c -o readfile` (рис. 15)

```
[guest@eavernikovskaya ~]$ gcc readfile.c -o readfile
[guest@eavernikovskaya ~]$ ls
Desktop  dir2  Documents  file4  Pictures  readfile  simpleid  simpleid2.c  Templates
dir1     dir3  Downloads  Music  Public   readfile.c  simpleid2  simpleid.c   Videos
```

Рис. 15: Компиляция программы `readfile.c`

Далее сменим владельца у файла `readfile.c` и изменим права так, чтобы только суперпользователь (`root`) мог прочитать его, а `guest` не мог (рис. 16)

```
[eavernikovskaya@eavernikovskaya ~]$ sudo chown root:guest /home/guest/readfile.c
[sudo] password for eavernikovskaya:
[eavernikovskaya@eavernikovskaya ~]$ sudo chmod u+s /home/guest/readfile.c
[eavernikovskaya@eavernikovskaya ~]$ sudo chmod 700 /home/guest/readfile.c
[eavernikovskaya@eavernikovskaya ~]$ sudo chmod -r /home/guest/readfile.c
[eavernikovskaya@eavernikovskaya ~]$ sudo chmod u+s /home/guest/readfile.c
[eavernikovskaya@eavernikovskaya ~]$ sudo ls -l /home/guest/readfile.c
--ws-----. 1 root guest 434 Apr 16 15:35 /home/guest/readfile.c
[eavernikovskaya@eavernikovskaya ~]$
```

Рис. 16: Смена владельца файла и прав доступа к файлу `readfile`

Проверим, что пользователь `guest` не может прочитать файл `readfile.c` (рис. 17)

```
[guest@eavernikovskaya ~]$ cat readfile.c  
cat: readfile.c: Permission denied  
[guest@eavernikovskaya ~]$
```

Рис. 17: Попытка прочесть содержимое файла `readfile.c`

Теперь попробуем прочесть эти же файлы от имени суперпользователя (рис. 20)

```
[eavernikovskaya@eavernikovskaya ~]$ sudo /home/guest/readfile /etc/shadow
root:$6$lj.3KPwiiHiHWivV$c.VerhCRKyCOefZG5QCmRag0HBeJ6YHXJzKysnwL4pc003lYIXxpF8R8P32wwlPeFSUD2Sp653ULY.ENYUCq0::0:999
99:7::
bin:!:19820:0:99999:7::
daemon:!:19820:0:99999:7::
adm:!:19820:0:99999:7::
lp:!:19820:0:99999:7::
svpc:!:19820:0:99999:7::
```

Рис. 20: Попытка прочесть содержимое файлов от имени суперпользователя

Выясним, установлен ли атрибут Sticky на директории /tmp, для чего выполним команду `ls -l / | grep tmp`. Так как в выводе есть буква t, это значит что атрибут установлен (рис. 21)

```
[guest@eavernikovskaya ~]$ ls -l / | grep tmp
drwxrwxrwt. 17 root root 4096 Apr 16 15:39 tmp
[guest@eavernikovskaya ~]$
```

Рис. 21: Проверка атрибутов директории tmp

От имени пользователя guest создадим файл file01.txt в директории /tmp со словом test: `echo "test" > /tmp/file01.txt` (рис. 22)

```
[guest@eavernikovskaya ~]$ echo "test" > /tmp/file01.txt  
[guest@eavernikovskaya ~]$
```

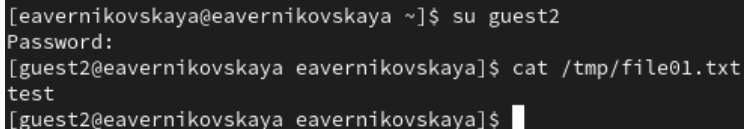
Рис. 22: Создание файла file01.txt со словом test

Просмотрим атрибуты у только что созданного файла `ls -l /tmp/file01.txt` и разрешим чтение и запись для категории пользователей «все остальные»:
`chmod o+rw /tmp/file01.txt` (рис. 23)

```
[guest@eavernikovskaya ~]$ ls -l /tmp/file01.txt
-rw-r--r--. 1 guest guest 5 Apr 16 15:43 /tmp/file01.txt
[guest@eavernikovskaya ~]$ chmod o+rw /tmp/file01.txt
[guest@eavernikovskaya ~]$ ls -l /tmp/file01.txt
-rw-r--rw-. 1 guest guest 5 Apr 16 15:43 /tmp/file01.txt
[guest@eavernikovskaya ~]$
```

Рис. 23: Смена атрибутов файла file01.txt

От пользователя `guest2` (не являющегося владельцем) попробуем прочитать файл `/tmp/file01.txt`: `cat /tmp/file01.txt` (рис. 24)

A terminal window with a dark background and light-colored text. The text shows a user at the prompt [eavernikovskaya@eavernikovskaya ~]\$ running the command 'su guest2'. The system prompts for a password, which is entered. The prompt then changes to [guest2@eavernikovskaya eavernikovskaya]\$. The user runs 'cat /tmp/file01.txt', and the output 'test' is displayed. The prompt returns to [guest2@eavernikovskaya eavernikovskaya]\$.

```
[eavernikovskaya@eavernikovskaya ~]$ su guest2
Password:
[guest2@eavernikovskaya eavernikovskaya]$ cat /tmp/file01.txt
test
[guest2@eavernikovskaya eavernikovskaya]$
```

Рис. 24: Попытка чтения файла

От пользователя `guest2` попробуем дозаписать в файл `/tmp/file01.txt` слово `test2` командой `echo "test2" > /tmp/file01.txt` (рис. 25)

```
[guest2@eavernikovskaya eavernikovskaya]$ echo "test2" > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
[guest2@eavernikovskaya eavernikovskaya]$ cat /tmp/file01.txt
test
[guest2@eavernikovskaya eavernikovskaya]$
```

Рис. 25: Попытка дозаписи в файл

От пользователя `guest2` попробуем записать в файл `/tmp/file01.txt` слово `test3`, стерев при этом всю имеющуюся в файле информацию командой `echo "test3" > /tmp/file01.txt` (рис. 26)

```
[guest2@eavernikovskaya eavernikovskaya]$ echo 'test3' > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
[guest2@eavernikovskaya eavernikovskaya]$ cat /tmp/file01.txt
test
[guest2@eavernikovskaya eavernikovskaya]$
```

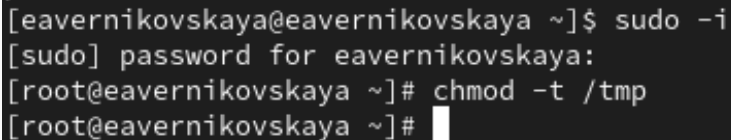
Рис. 26: Попытка записи в файл и удаление всей имеющейся в нём информации

От пользователя `guest2` попробуем удалить файл `/tmp/file01.txt` командой `rm /tmp/file01.txt` (рис. 27)

```
[guest2@eavernikovskaya eavernikovskaya]$ rm /tmp/file01.txt
rm: remove write-protected regular file '/tmp/file01.txt'? y
rm: cannot remove '/tmp/file01.txt': Operation not permitted
[guest2@eavernikovskaya eavernikovskaya]$
```

Рис. 27: Попытка удалить файл

Далее от имени суперпользователя снимем с директории `tmp` атрибут Sticky командой `chmod -t /tmp` (рис. 28), (рис. 29)



```
[eavernikovskaya@eavernikovskaya ~]$ sudo -i  
[sudo] password for eavernikovskaya:  
[root@eavernikovskaya ~]# chmod -t /tmp  
[root@eavernikovskaya ~]#
```

Рис. 28: Снятие с директории `tmp` атрибута Sticky


```
[guest2@eavernikovskaya eavernikovskaya]$ ls -l / | grep tmp  
drwxrwxrwx. 17 root root 4096 Apr 16 15:50 tmp  
[guest2@eavernikovskaya eavernikovskaya]$
```

Рис. 29: Проверка того что атрибут снялся

Далее снова проделаем предыдущие шаги. Всё то же самое только теперь можно удалить файл (рис. 30)

```
[guest2@eavernikovskaya eavernikovskaya]$ cat /tmp/file01.txt
test
[guest2@eavernikovskaya eavernikovskaya]$ echo "test2" > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
[guest2@eavernikovskaya eavernikovskaya]$ cat /tmp/file01.txt
test
[guest2@eavernikovskaya eavernikovskaya]$ echo "test3" > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
[guest2@eavernikovskaya eavernikovskaya]$ cat /tmp/file01.txt
test
[guest2@eavernikovskaya eavernikovskaya]$ rm /tmp/file01.txt
rm: remove write-protected regular file '/tmp/file01.txt'? y
[guest2@eavernikovskaya eavernikovskaya]$
```

Рис. 30: Повтор предыдущих действий после снятия атрибута Sticky

Вернём директории tmp атрибут t от имени суперпользователя (рис. 31)

```
[root@eavernikovskaya ~]# chmod +t /tmp
[root@eavernikovskaya ~]# ls -l / | grep tmp
drwxrwxrwt. 17 root root 4096 Apr 16 15:52 tmp
[root@eavernikovskaya ~]#
```

Рис. 31: Изменение атрибутов

Подведение итогов

В результате выполнения работы мы изучили механизмы изменения идентификаторов, применения SetUID- и Sticky-битов. Получили практические навыки работы в консоли с дополнительными атрибутами. Рассмотрели работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов

1. Лабораторная работа №5 [Электронный ресурс] URL:
https://esystem.rudn.ru/pluginfile.php/2580984/mod_resource/content/2/005-lab_discret_sticky.pdf
2. Инструментарий программиста в Linux: Компилятор GCC [Электронный ресурс] URL: <http://parallel.imm.uran.ru/freesoft/make/instrum.html>